

```

# =====
# SET OPERATIONS AND VENN DIAGRAM
# Menu-driven program
# =====

import matplotlib.pyplot as plt
from matplotlib_venn import venn2, venn3

# =====
# INPUT VALIDATION
# =====

def get_positive_integer(message):
    while True:
        try:
            value = int(input(message))

            if value > 0:
                return value
            else:
                print("Please enter a positive integer.")

        except ValueError:
            print("Invalid input. Please enter a number.")

def get_set_input(set_number):
    while True:
        elements = input(
            f"Enter elements of Set {set_number}, separated by spaces: "
        ).strip()

        if elements == "":
            print("Set cannot be empty. Please enter at least one element.")
            continue

        # Convert input into a set
        current_set = set(elements.split())

        if len(current_set) == 0:
            print("Invalid set. Please try again.")
        else:
            return current_set

# =====
# ENTER SETS
# =====

def enter_sets():
    number_of_sets = get_positive_integer(
        "Enter the number of sets (2 or 3): "
    )

    while number_of_sets not in [2, 3]:
        print("For Venn diagram visualization, enter either 2 or 3 sets.")
        number_of_sets = get_positive_integer(
            "Enter the number of sets (2 or 3): "
        )

    sets = []

    for i in range(1, number_of_sets + 1):
        current_set = get_set_input(i)
        sets.append(current_set)

    return sets

# =====
# UNIVERSAL SET
# =====

def enter_universal_set():
    while True:
        elements = input(
            "Enter Universal Set elements, separated by spaces: "
        ).strip()

        if elements == "":

```

```

    print("Universal Set cannot be empty.")
    continue

    universal_set = set(elements.split())

    if len(universal_set) == 0:
        print("Invalid Universal Set.")
    else:
        return universal_set

# =====
# SET OPERATIONS
# =====

def perform_operation(sets, universal_set=None):

    print("\n===== SET OPERATIONS =====")
    print("1. Union")
    print("2. Intersection")
    print("3. Difference")
    print("4. Symmetric Difference")
    print("5. Complement")
    print("6. Back to Main Menu")

    choice = input("Enter your choice: ").strip()

    # -----
    # UNION
    # -----

    if choice == "1":

        result = sets[0]

        for current_set in sets[1:]:
            result = result.union(current_set)

        print("\nUnion =", result)

        return "Union", result

    # -----
    # INTERSECTION
    # -----

    elif choice == "2":

        result = sets[0]

        for current_set in sets[1:]:
            result = result.intersection(current_set)

        print("\nIntersection =", result)

        return "Intersection", result

    # -----
    # DIFFERENCE
    # -----

    elif choice == "3":

        print("\nAvailable Sets:")

        for i in range(len(sets)):
            print(f"{i + 1}. Set {i + 1}")

        first = get_positive_integer(
            "Enter first set number: "
        )

        second = get_positive_integer(
            "Enter second set number: "
        )

        if first > len(sets) or second > len(sets):
            print("Invalid set number.")
            return None, None

        if first == second:
            print("Please select two different sets.")

```

```

        return None, None

    result = sets[first - 1].difference(sets[second - 1])

    print(
        f"\nSet {first} - Set {second} =",
        result
    )

    return "Difference", result

# -----
# SYMMETRIC DIFFERENCE
# -----

elif choice == "4":

    if len(sets) != 2:
        print(
            "Symmetric Difference visualization is "
            "implemented for two sets."
        )

    result = sets[0].symmetric_difference(sets[1])

    print("\nSymmetric Difference =", result)

    return "Symmetric Difference", result

# -----
# COMPLEMENT
# -----

elif choice == "5":

    if universal_set is None:
        print("\nUniversal Set is required for Complement.")
        universal_set = enter_universal_set()

    print("\nAvailable Sets:")

    for i in range(len(sets)):
        print(f"{i + 1}. Set {i + 1}")

    set_number = get_positive_integer(
        "Enter set number for complement: "
    )

    if set_number > len(sets):
        print("Invalid set number.")
        return None, None

    result = universal_set.difference(
        sets[set_number - 1]
    )

    print(
        f"\nComplement of Set {set_number} =",
        result
    )

    return "Complement", result

elif choice == "6":
    return None, None

else:
    print("Invalid choice.")
    return None, None

# =====
# VENN DIAGRAM
# =====

def display_venn(sets, operation):

    if len(sets) == 2:

        set_a = sets[0]
        set_b = sets[1]

        plt.figure(figsize=(7, 6))

```

```

    diagram = venn2(
        [set_a, set_b],
        set_labels=("Set A", "Set B")
    )

    # Show values in the diagram
    if diagram.get_label_by_id("10"):
        diagram.get_label_by_id("10").set_text(
            "\n".join(sorted(set_a - set_b))
        )

    if diagram.get_label_by_id("01"):
        diagram.get_label_by_id("01").set_text(
            "\n".join(sorted(set_b - set_a))
        )

    if diagram.get_label_by_id("11"):
        diagram.get_label_by_id("11").set_text(
            "\n".join(sorted(set_a & set_b))
        )

    plt.title(f"Venn Diagram - {operation}")
    plt.show()

elif len(sets) == 3:

    set_a = sets[0]
    set_b = sets[1]
    set_c = sets[2]

    plt.figure(figsize=(8, 7))

    diagram = venn3(
        [set_a, set_b, set_c],
        set_labels=("Set A", "Set B", "Set C")
    )

    # Display elements in each region
    regions = {
        "100": set_a - set_b - set_c,
        "010": set_b - set_a - set_c,
        "001": set_c - set_a - set_b,
        "110": (set_a & set_b) - set_c,
        "101": (set_a & set_c) - set_b,
        "011": (set_b & set_c) - set_a,
        "111": set_a & set_b & set_c
    }

    for region_id, values in regions.items():

        label = diagram.get_label_by_id(region_id)

        if label:
            label.set_text(
                "\n".join(sorted(values))
            )

    plt.title(f"Venn Diagram - {operation}")
    plt.show()

# =====
# MAIN MENU
# =====

def main():

    sets = []
    universal_set = None

    while True:

        print("\n")
        print("=====")
        print("          SET OPERATIONS AND VENN DIAGRAM")
        print("=====")
        print("1. Enter Sets")
        print("2. Enter Universal Set")
        print("3. Perform Set Operation")
        print("4. Display Current Sets")
        print("5. Exit")
        print("=====")

```

```

choice = input("Enter your choice: ").strip()

# -----
# ENTER SETS
# -----

if choice == "1":

    sets = enter_sets()

    print("\nSets entered successfully.")

    for i, current_set in enumerate(sets):
        print(f"Set {i + 1} =", current_set)

# -----
# ENTER UNIVERSAL SET
# -----

elif choice == "2":

    universal_set = enter_universal_set()

    print(
        "\nUniversal Set =",
        universal_set
    )

# -----
# PERFORM OPERATION
# -----

elif choice == "3":

    if len(sets) < 2:
        print(
            "\nPlease enter at least two sets first."
        )
        continue

    operation, result = perform_operation(
        sets,
        universal_set
    )

    if operation is not None:

        # Ask whether to display Venn diagram
        while True:

            answer = input(
                "\nDo you want to view the Venn diagram? "
                "(yes/no): "
            ).strip().lower()

            if answer in ["yes", "y"]:
                display_venn(
                    sets,
                    operation
                )
                break

            elif answer in ["no", "n"]:
                print("Venn diagram skipped.")
                break

            else:
                print(
                    "Invalid input. Enter yes or no."
                )

# -----
# DISPLAY SETS
# -----

elif choice == "4":

    if len(sets) == 0:
        print("\nNo sets have been entered.")

    else:

```

```
        print("\nCurrent Sets:")

        for i, current_set in enumerate(sets):
            print(
                f"Set {i + 1} =",
                current_set
            )

        if universal_set is not None:
            print(
                "Universal Set =",
                universal_set
            )

# -----
# EXIT
# -----

elif choice == "5":

    print("\nProgram terminated.")
    break

else:

    print(
        "\nInvalid choice. Please select "
        "a valid menu option."
    )

# =====
# RUN PROGRAM
# =====

main()
```

```
=====
      SET OPERATIONS AND VENN DIAGRAM
=====
1. Enter Sets
2. Enter Universal Set
3. Perform Set Operation
4. Display Current Sets
5. Exit
=====
Enter your choice: 1
Enter the number of sets (2 or 3): 3
Enter elements of Set 1, separated by spaces: 1 2 3 4 5
Enter elements of Set 2, separated by spaces: 2 3 4 5 6
Enter elements of Set 3, separated by spaces: 3 4 5 6 7

Sets entered successfully.
Set 1 = {'1', '3', '2', '4', '5'}
Set 2 = {'6', '3', '2', '4', '5'}
Set 3 = {'6', '3', '4', '5', '7'}
```

```
=====
      SET OPERATIONS AND VENN DIAGRAM
=====
1. Enter Sets
2. Enter Universal Set
3. Perform Set Operation
4. Display Current Sets
5. Exit
=====
Enter your choice: 3

===== SET OPERATIONS =====
1. Union
2. Intersection
3. Difference
4. Symmetric Difference
5. Complement
6. Back to Main Menu
Enter your choice: 2
```

Intersection = {'4', '5', '3'}

Do you want to view the Venn diagram? (yes/no): yes

/usr/local/lib/python3.13/dist-packages/matplotlib_venn/layout/venn3/pairwise.py:169: UserWarning: Bad circle positioning.
warnings.warn("Bad circle positioning.")

Venn Diagram - Intersection

